

Grille du barème (80 pts)

Exercice: (4 pts)

Q.1 : 0.5 pt

def affiche (X):	
for e in X :	0.25
print(e)	0.25

Q.2 : 0.75 pt

def somme (X) :	
S=0 for e in X : S=S+e return S	0.75

Q.3 : 0.25 pt

Complexité linéaire $O(n)$ avec $n=len(X)$ Justification : boucle	0.25
--	------

Q.4 : 0.25 pt

def moyenne (X) :	
return somme(X)/len(X)	0.25

Q.5 : 1 pt

def compte (X) :	
c, m=0, moyenne(X) for e in X : if e>m : c=c+1 return c	1

Q.6 : 1 pt

def sigma (X) :	
S, m=0, moyenne(X) for e in X : S=S+(e-m)**2 return S	1

Q.7 : 0.25 pt

def ecart_type (X) :	
return (sigma(X)/len(X))**0.5	0.25

Partie I : Base de données et langage SQL (20 pts)

Q.1 : 1.5 pt

Update Tournoi Set catégorie='WTA 1000'	0.5
Where catégorie in ('WTA Premier Mandatory' , 'WTA Premier 5')	1

Q.2 : 2.5 pts

Pi pays (Sigma catégorie=' ATP 500' (Tournois))	1
intersection	1
Pi pays (Sigma catégorie=' Master 1000' (Tournois))	0.5

Q.3 : 2.5 pts

Select pays From Tournois Where catégorie=' ATP 500' intersect Select pays From Tournois Where catégorie='Master 1000'	2.5
--	------------

Q.4 : 2.5 pts

Select catégorie, count(*)	1
From Tournois	0.25
Group by catégorie	1
Order by catégorie	0.25

Q.5 : 4 pts

Select nomJr, genre, max(prime), min(prime)	1
From Joueur join Gagne on id=idJr	0.5
Group by id	1
Having sum(prime) >75	1
Order by avg(prime) Desc	0.5

Q.6 : 4 pts

Select année, nomJr, genre, nationalité	0.25
From (Joueur join Gagne on id =idJr) join Tournoi on codTr =code	1
Where catégorie='Grand Chelem'	0.5
Group by id, année	1
Having count(code)= 4	1
Order by année desc	0.25

Q.7 : 3 pts

Select A.codTr , A.prime, B.prime	1
From Gagne A join Gagne B on A.codTr =B.codTr	1
Where A.année=2020 and B.année=2021	1

Partie II: Calcul numérique (20 pts)**Q. 1 : 1.5 pts**

def matrice_zeros (n) :	
return np.zeros ((n, n))	1.5

Q. 2 : 1.5 pts

def matrice_id (n) :	
return np.eye (n)	1.5

Q. 3 : 3 pts

def sigma(L, U, i, j) :	
S=0	0.5
for k in range (i):	1
S = S + L[i, k] * U[k, j]	1
return S	0.5

Q. 4 : 4.5 pts

def Doolittle (M) :	
n=len (M) U=matrice_zeros (n) L=matrice_id (n)	1
for i in range (n):	0.5
for j in range (i, n): U[i, j] = M[i, j] – sigma (L, U, i, j)	1
for j in range(i+1,n): L[j, i] = (M[j, i] – sigma (L, U, j, i)) / U[i, i]	1.5
return L, U	0.5

Q. 5 : 4.5 pts

def descente (L, Y) :	
n=len (L) Z=np.zeros (n)	0.5
for i in range (n) :	0.5
S=0 for j in range (i) : S = S + L[i, j]*Z[j]	2
Z[i]=Y[i]-S	1
return Z	0.5

Q. 6 : 5 pts

def remonte (U, Z) :	
n=len (U) X=np.zeros (n)	0.5
for i in range (n-1, -1, -1) :	1
S=0 for j in range (n-1, i, -1) : S = S + U[i, j]*X[j]	1.5
X[i]=(Z[i]-S)/U[i, i]	1.5
return X	0.5

Partie III: Problème Python (40 pts)**Q.1 : 4 pts**

def tri_sequence (S) : n=len(S) for i in range(n-1) : k=i for j in range(i+1, n) : if S[j]<S[k]: k=j S[i],S[k]=S[k],S[i]	4	S.sort() 1
--	---	-------------------

Q.2 : 5 pts

def sous_sequence (X, Y) :	
p=0 for e in X : if e not in Y[p:] : return False else: k=Y[p:].index(e) p=p+k return True	5

Q.3 : 4.5 pts

def taille_PLSSC(X,Y) :	
if len(X)==0 or len(Y)==0: return 0	1.5
elif X[-1]==Y[-1]: return 1+taille_PLSSC(X[:-1],Y[:-1])	1.5
else: return max(taille_PLSSC(X[:-1],Y),taille_PLSSC(X,Y[:-1]))	1.5

Q.4.a : 3 pts

def matrice_nulle (n,p) :	
M=[] for i in range(n+1) : M.append([0]*(p+1)) return M	3

Q.4.b : 5 pts

def matrice_tailles (X, Y) :	
n,p=len(X),len(Y) M=matrice_nulle (n, p)	1
for i in range(1, n+1): for j in range(1, p+1):	2
if X[i-1]==Y[j-1]: M[i][j]=M[i-1][j-1]+1 else: M[i][j]=max(M[i][j-1],M[i-1][j])	2
return M	0.5

Q. 4.c : 1.5 pts

def taille_plssc (X, Y) :	
M=matrice_tailles(X, Y)	1
return M[-1][-1]	0.5

Q. 4.d : 2 pts

Complexité quadratique : $O(\text{len}(X)*\text{len}(Y))$	1
Deux boucles imbriquées	1

Q. 5 : 6 pts

def plssc(X, Y) :	
M=matrice_tailles(X, Y)	0.5
i, j, L=len(X), len(Y), []	0.5
while i!=0 and j!=0:	1
if X[i-1]==Y[j-1]: L=[X[i-1]]+L i,j=i-1,j-1	1.5
elif M[i-1][j]>=M[i][j-1]: i=i-1 else: j=j-1	2
return L	0.5

Q. 6 : 6 pts

def lcs (X,Y):	
def plss(X,Y,i,j,S) : if i==0 or j==0: if S not in L: L.append(S) elif X[i-1]==Y[j-1]: plss(X,Y,i-1,j-1,[X[i-1]]+S) elif M[i][j-1]==M[i-1][j]: plss(X,Y,i-1,j,S) plss(X,Y,i,j-1,S) elif M[i-1][j]<M[i][j-1]: plss(X,Y,i,j-1,S) else: plss(X,Y,i-1,j,S)	6
M=matrice_tailles(X,Y) i,j=len(X),len(Y) S,L=[],[] plss(X,Y,i,j,S) return L	

Q. 7 : 3 pts

def plss_croissantes (S) :	
C=S[:] tri_sequence(C)	2
return lcs(S, C)	1